

Spring Data JPA Queries: Styles, Best Practices, and Examples

Executive Summary: Spring Data JPA offers multiple query styles to cover simple and complex data access needs. Derived query methods (method-name conventions) and annotated `@Query` methods with JPQL provide quick, declarative queries. Native SQL queries allow maximum flexibility at the cost of portability. For dynamic or programmatic queries, the JPA **Criteria API**, **Querydsl**, **Specifications**, and **Query by Example** (QBE) offer powerful alternatives. **EntityGraphs** and **JOIN FETCH** help avoid the N+1 select problem by eagerly fetching associations. **Projections** (interface- or class-based) let repositories return partial views (DTOs) for efficiency. Pagination and sorting are first-class via `Pageable` and `Sort`. Each style has trade-offs: derived methods are concise but limited; JPQL is expressive but string-based; native queries are fast but DB-specific; Criteria/Querydsl and Specifications are type-safe but verbose; QBE is simple but limited; EntityGraphs improve performance but require upfront design; projections reduce data but risk complexity.

Parameter binding in Spring Data JPA defaults to positional parameters but can (and should) use `@Param` or Java 8 parameter name discovery for clarity [24†L687-L694]. Repository return types include collections (`List`, `Streamable`), `Optional`, `Stream`, and paging types (`Page`, `Slice`, `Window`) [20†L179-L187]. Transactionally, Spring Data applies read-only transactions to finder methods by default and plain transactions to modifying methods [12†L93-L100]; `@Transactional` and `@Modifying` can override this (see **Bulk Updates** example below). Locking via `@Lock` applies JPA lock modes to queries [11†L93-L102]. Performance must consider query efficiency, caching, and common pitfalls like the N+1 problem. First-level (session) caching is automatic; a configurable second-level cache (e.g. Ehcache) can be enabled via `@Cacheable` [37†L49-L57] [36†L259-L268]. To combat N+1 issues, fetch associations eagerly (e.g. **JOIN FETCH** or `@EntityGraph`) when needed [9†L1121-L1130] [16†L197-L202]. Testing repositories is typically done with Spring Boot's `@DataJpaTest`, which spins up an in-memory database and rolls back each test's transaction [31†L84-L90] [32†L257-L260].

Across Spring Data versions (latest Spring Data JPA 4.x/Spring Boot 3.x assumed), query API remains stable. Be aware of Jakarta namespace changes (`jakarta.persistence.*` instead of `javax.persistence`), and optional Querydsl (via the community fork) requiring annotation processing setup [25†L148-L156] [24†L687-L694]. Always refer to official release notes for migration details [13†L93-L100].

This report details each query style with explanations, runnable Java/Spring Boot examples, best practices, and references. Tables compare features and guide style selection. Entity-relationship and decision flow diagrams are provided using Mermaid. Code snippets are ready for Java 17+ with Spring Boot.

Supported Query Styles in Spring Data JPA

1. Derived Query Methods

Spring Data derives SQL/JPQL from method names. You simply declare a method like `findByPropertyAndOtherProperty(...)`, and Spring Data builds a JPQL query. For example:

```
public interface UserRepository extends Repository<User, Long> {  
    // Derived query method by method name  
    List<User> findByEmailAddressAndLastname(String emailAddress, String  
    lastname);  
}
```

Spring translates this to JPQL: `SELECT u FROM User u WHERE u.emailAddress = ?1 AND u.lastname = ?2` [1†L131-L134]. Many keywords are supported (e.g. `And`, `Or`, `Between`, `LessThan`, `OrderBy`, `Distinct`, etc.) [39†L147-L156] [39†L250-L259]. For example:

```
List<User> findByAgeGreaterThanOrderByLastnameDesc(int age);
```

generates `... WHERE x.age > ?1 ORDER BY x.lastname DESC` [39†L172-L181] [39†L250-L259].

Pitfalls: Long method names can become unreadable or exceed parser capability. Also, invalid property names only surface as exceptions at startup or runtime. For example, renaming `findByFoo()` on an entity with no `foo` property triggers a `PropertyReferenceException` [31†L144-L152].

Best Practices: Keep method names concise and meaningful. When methods get very long or complex, consider using `@Query` with JPQL or refactoring logic. Test queries, especially complex ones (see Testing section).

When to Use: For simple queries (filter by one or two fields) where performance is not extreme. Derived methods are quick to write and automatically validated at application startup [31†L144-L152].

2. @Query with JPQL

When derived method names aren't flexible enough, annotate methods with `@Query`. Write JPQL (entity-oriented SQL) directly:

```
public interface UserRepository extends JpaRepository<User, Long> {  
    // Custom JPQL query with named parameters  
    @Query("SELECT u FROM User u WHERE u.firstname = :name OR u.lastname  
    = :name")  
    User findByNameOrNickname(@Param("name") String name);  
}
```

Here `:name` binds to the `name` parameter via `@Param` [24†L687-L694]. Spring Data defaults to position-based binding (e.g. `?1`), but named parameters with `@Param` are safer, and Java 8 `parameters` can auto-discover names [24†L687-L694].

Pitfalls: JPQL queries are written as strings, so typos won't be caught until runtime (or startup when using Hibernate's parser). Complex JPQL can be cumbersome. Be mindful of entity names (`#{#entityName}` can avoid hard-coding) [24†L745-L753].

Best Practices: Use `@Param` for clarity; take advantage of SpEL expressions for flexibility (e.g. `#{#entityName}`) [24†L745-L754]. Test queries to ensure correctness.

When to Use: For queries that cannot be expressed by method names, such as joins, aggregations, or complex `WHERE` logic. Use JPQL when portability is needed but you need more control than derived methods.

3. Native SQL Queries

Spring Data allows native SQL queries via `@Query(nativeQuery = true)`. For example:

```
public interface OrderRepository extends JpaRepository<Order, Long> {
    @Query(value = "SELECT * FROM orders o WHERE o.customer_id = :custId",
        nativeQuery = true)
    List<Order> findByCustomerNative(@Param("custId") Long custId);
}
```

This runs raw SQL against the database. Native queries are database-specific and bypass JPQL parsing.

Pitfalls: Spring/Hibernate **do not validate** native queries at startup [31†L198-L202]. An invalid SQL will only fail at runtime. Results must match entity mappings, or you must use result-set mappings or constructor projections for DTOs. Native queries might not work with in-memory DBs in tests if SQL dialect differs.

Best Practices: Use native queries sparingly, only when necessary (e.g. for vendor-specific features or complex queries not expressible in JPQL). Always test them in an environment matching production DB. You can map to DTOs via `SqlResultSetMapping` or by selecting into constructor expressions.

When to Use: When JPQL cannot express a needed query (e.g. using window functions, vendor-specific syntax) or for complex performance-critical queries. Otherwise, prefer JPQL for portability.

4. Criteria API

The JPA Criteria API (`CriteriaBuilder`, `CriteriaQuery`) builds type-safe, dynamic queries in code (no string JPQL). For example:

```
@Repository
public class BookDao {
    @PersistenceContext
    private EntityManager em;
```

```

    public List<Book> findBooksByAuthorAndTitle(String authorName, String
titleSubstr) {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Book> cq = cb.createQuery(Book.class);
        Root<Book> book = cq.from(Book.class);

        Predicate authorPred = cb.equal(book.get("author"), authorName);
        Predicate titlePred = cb.like(book.get("title"), "%" + titleSubstr +
"%");
        cq.where(cb.and(authorPred, titlePred));

        return em.createQuery(cq).getResultList();
    }
}

```

This builds a `SELECT b FROM Book b WHERE b.author = ?1 AND b.title LIKE ?2` query at runtime [\[27†L289-L297\]](#) .

Pitfalls: Criteria code is verbose and harder to read. Long predicates require careful parenthesization (Criteria helps with `and`, `or`).

Best Practices: Use Criteria when you need dynamic query composition (adding predicates conditionally). Using the generated static metamodel (e.g. `Book_` fields) avoids string literals in `get("...")` .

When to Use: For complex or dynamic queries not known at compile time, especially when safety of field names is desired. Good for programmatically building queries (e.g. based on UI filters).

5. Querydsl

Querydsl is a third-party (now community-forked) library for type-safe queries via generated Q-type classes. Spring Data supports Querydsl via `QuerydslPredicateExecutor` . Example:

```

interface UserRepository extends CrudRepository<User, Long>,
QuerydslPredicateExecutor<User> {}

public void exampleUsage(UserRepository userRepository) {
    QUser u = QUser.user;
    Predicate predicate = u.firstname.equalsIgnoreCase("Alice")
        .and(u.age.goe(18));
    Iterable<User> adultsNamedAlice = userRepository.findAll(predicate);
}

```

Here `QUser` is a generated metamodel (needs annotation processing setup) and predicates are composable in Java [\[25†L131-L139\]](#) [\[25†L143-L146\]](#) .

Pitfalls: Requires extra dependencies (`querydsl-jpa`) and build config for annotation processing to generate Q-classes [\[25†L148-L156\]](#) . Learning curve can be steep if unfamiliar with the fluent API.

Best Practices: Ensure Q-types are generated (e.g. Maven APT plugin or Gradle annotation processor). Leverage predicate composition (`and` , `or`) for dynamic filters.

When to Use: When you need very type-safe, compile-checked, dynamic queries with complex logical combinations. Ideal for applications with many conditional search criteria.

6. Specifications

Spring Data's `Specification` wraps the JPA Criteria API into a reusable predicate interface. A `Specification<T>` is a predicate over `T`. For example:

```
public class CustomerSpecs {
    public static Specification<Customer> hasName(String name) {
        return (root, query, builder) ->
            builder.equal(root.get("name"), name);
    }
    public static Specification<Customer> ageGreaterThan(int age) {
        return (root, query, builder) ->
            builder.greaterThan(root.get("age"), age);
    }
}

// Usage:
List<Customer> result = customerRepository.findAll(
    Specification.where(CustomerSpecs.hasName("Alice"))
        .and(CustomerSpecs.ageGreaterThan(30))
);
```

This yields a query like

`SELECT c FROM Customer c WHERE c.name = 'Alice' AND c.age > 30`. Specifications can be composed with `.and()`, `.or()` [\[4†L153-L162\]](#) .

Pitfalls: Still relies on Criteria and is verbose to write. All combinations must be covered by code.

Best Practices: Define small, reusable `Specification` methods (as above). Combine them based on user inputs. Use static metamodel (`Customer_`) for field references if available [\[4†L146-L154\]](#) .

When to Use: For dynamic filtering with many optional conditions (e.g. search forms). Keeps repository interface clean (just extend `JpaSpecificationExecutor`) and pushes complexity into spec classes.

7. Query by Example (ExampleMatcher)

QBE lets you pass a “probe” entity and `ExampleMatcher` to build queries:

```
Person probe = new Person();
probe.setFirstname("Alice");
ExampleMatcher matcher = ExampleMatcher.matching()
    .withIgnorePaths("id")
```

```

        .withMatcher("lastname",
ExampleMatcher.GenericPropertyMatchers.contains().ignoreCase());
Example<Person> example = Example.of(probe, matcher);

List<Person> results = personRepository.findAll(example);

```

This finds persons with `firstname=Alice` (exact) and `lastname` containing some value (if set on probe). By default, null-valued fields are ignored [\[3†L164-L172\]](#) .

Pitfalls: Very limited: no ORs or groups, no joins, no collections, and matching is per-property only [\[3†L134-L142\]](#) . You can't express complex logic.

Best Practices: Use for simple dynamic filters that mirror an example object. Configure string matching (contains/startsWith) with `ExampleMatcher` .

When to Use: When you want quick "findBy example" behavior without writing queries, typically for form-based filters where an entity-like instance holds search criteria.

8. Entity Graphs (`@EntityGraph`)

`@EntityGraph` controls JPA fetch plans (eager vs lazy) on queries to address performance (e.g. N+1 selects). Define a graph on the entity:

```

@Entity
@NamedEntityGraph(name = "User.detail",
    attributeNodes = @NamedAttributeNode("roles"))
public class User {
    @OneToMany(fetch = FetchType.LAZY) List<Role> roles;
    // ...
}

```

Then use it on a repository method:

```

public interface UserRepo extends JpaRepository<User, Long> {
    @EntityGraph(value = "User.detail", type = EntityGraphType.LOAD)
    Optional<User> findById(Long id);
}

```

This tells JPA to fetch `roles` in the same query (using a join) [\[9†L1107-L1115\]](#) [\[9†L1127-L1131\]](#) . You can also define ad-hoc graphs:

```

@EntityGraph(attributePaths = {"roles"})
List<User> findByActiveTrue();

```

This inline graph fetches `roles` without `@NamedEntityGraph` [\[9†L1133-L1142\]](#) .

Pitfalls: Over-fetching can occur (fetching large collections). Use carefully; combining with pagination can produce Cartesian results. Also, only one `@EntityGraph` allowed per method.

Best Practices: Use to avoid N+1 when you know you will need associations. Define named graphs on entities and reuse them. Alternatively, use `JOIN FETCH` in JPQL for one-off cases.

When to Use: When lazy associations would trigger N+1 queries, and you want to load them eagerly in the same query. Especially useful for many-to-one or one-to-many lists you know will be accessed.

9. Projections

Projections limit the data retrieved by returning DTOs or partial views. Two main types are supported:

- **Interface-based projections:** Define an interface with getters matching entity properties:

```
interface NamesOnly {
    String getFirstname();
    String getLastName();
}
```

Then repository can return this projection:

```
interface PersonRepository extends JpaRepository<Person, UUID> {
    Collection<NamesOnly> findByLastname(String lastname);
}
```

Spring will fetch only the requested fields and provide proxy objects [\[5†L137-L147\]](#) .

- **Class-based (DTO) projections:** Define a class with a constructor matching selected fields, and use `new` in JPQL:

```
public class PersonDTO {
    private final String name;
    public PersonDTO(String name) { this.name = name; }
    // getter
}

interface PersonRepository extends JpaRepository<Person, Long> {

    @Query("SELECT new com.example.PersonDTO(p.firstname) FROM Person p WHERE p.active = true")
    List<PersonDTO> findActivePersonNames();
}
```

Pitfalls: Interface projections only work if method overrides do not conflict with base methods, and cannot be applied on `CrudRepository` base methods [\[5†L160-L167\]](#) . Also, missing properties or mismatched types cause errors. Class-based projections require constructor expressions in JPQL (no compile-time checking).

Best Practices: Prefer interface projections for simplicity if you need a subset of fields. Use nested interfaces for deep projections (e.g. address) [5†L173-L182] . For DTOs, ensure package name is correct and use `new` in JPQL.

When to Use: When you want to return only certain fields (e.g. for performance or to shape results). Good for API responses to avoid sending whole entities.

10. Pagination and Sorting

Spring Data JPA supports `Pageable` and `Sort` parameters:

```
Page<User> findByLastname(String lastname, Pageable pageable);
List<User> findByActiveTrue(Sort sort);
```

Callers can pass `PageRequest.of(page, size, Sort.by("name").descending())`. The repository returns a `Page<User>` (with total count) [20†L179-L187] or a `List` sorted accordingly. Internally, Spring adds `LIMIT/OFFSET` and `ORDER BY`.

Pitfalls: `Page` triggers an extra count query. Deep pagination (high page numbers) can be slow. Keyset pagination (scrolling or `Window` query) may be needed for very large data sets.

Best Practices: Use reasonable page sizes. If only checking existence of more pages, consider `Slice` or `Window` APIs. Always include an explicit `OrderBy` in page sorts to ensure consistent results.

When to Use: For any query returning potentially many rows where you want to fetch in chunks or only a page at a time (e.g. UI tables, APIs).

11. Parameter Binding and Return Types

- **Parameter Binding:** By default Spring Data uses positional binding (?1 , ?2) in JPQL examples. As noted, named parameters via `@Param("name")` provide clarity [24†L687-L694] . SpEL expressions (#{...}) can also be used (e.g. `#{#entityName}` for generic repos [24†L745-L754] , or to clean inputs [24†L801-L809]).
- **Return Types:** Supported return types include `void`, primitives, entities (`T`), `Optional<T>`, `List<T>`, `Iterable<T>`, `Stream<T>`, `Streamable<T>`, `Future<T>` / `CompletableFuture<T>` (with `@Async`), `Slice<T>`, `Page<T>`, `Window<T>` [20†L179-L187] . `Page` / `Slice` require a `Pageable` parameter. `Stream` results should be used with caution (close stream or use `try-with-resources`).

For example, a repository could have:

```
Optional<User> findByEmail(String email);
Stream<User> findByActiveTrueOrderByLastnameAsc();
Page<User> findByActiveTrue(Pageable pageable);
```

These return an entity, a Java stream, and a `Page` respectively [20†L179-L187] .

Transactional and Locking Behavior

By default, CRUD repository methods have transactions: read-only for finders and non-read-only for save/delete [12†L93-L100] . For example, `findAll()` runs in a read-only transaction unless overridden. To customize:

```
public interface UserRepository extends CrudRepository<User, Long> {  
    @Override  
    @Transactional(timeout = 5)  
    List<User> findAll(); // customized transaction (not read-only now)  
}
```

Repository methods with `@Modifying` (for `UPDATE` / `DELETE` queries) must be annotated with `@Transactional` as well [29†L175-L178] :

```
@Modifying @Transactional  
@Query("DELETE FROM User u WHERE u.active = false")  
void deleteInactiveUsers(); // runs with readOnly=false because of @Modifying
```

This overrides the default read-only flag [29†L175-L184] .

For **locking**, use the JPA `@Lock` annotation on methods to apply lock modes:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)  
Optional<User> findWithLockById(Long id);
```

This attaches a lock (e.g. `SELECT ... FOR UPDATE`) to prevent concurrent updates [11†L93-L102] .

Performance Considerations & Caching

- **First-level (Session) Cache:** Every `EntityManager` (per transaction) caches entities, so repeated `findById` of the same entity hits the L1 cache.
- **Second-level Cache:** Optional; configure a provider (Ehcache, Hazelcast, etc.) and annotate entities with `@Cacheable(true)` to share across sessions [37†L49-L57] . This avoids repeated DB hits for entities.
- **Query Cache:** Hibernate can cache query results if enabled, but Spring Data itself has no built-in query caching. You can set JPA hints (`@QueryHints`) if needed.
- **N+1 Problem:** If you lazy-load a collection in a loop, JPA may issue one query for the list and N additional queries for each item. For example, fetching a list of `Author` and then calling `author.getPosts()` in a loop triggers N+1 selects. Mitigate by using `JOIN FETCH` in JPQL or `@EntityGraph` to fetch associations in one query [16†L197-L202] [9†L1121-L1130] .

- **Bulk Updates:** Bulk `@Modifying` JPQL/SQL queries bypass persistence context and can cause inconsistency if entities are cached. It's best to use them sparingly and ensure entities are refreshed if needed.
- **Pagination:** Deep pagination can be slow; consider keyset scrolling (`Window` API in Spring Data) for large datasets.
- **Indexing:** Ensure database indexes match frequent query filters.

N+1 Problem and Solutions

The N+1 SELECT problem occurs when loading an entity and its lazy associations. For example:

```
List<Author> authors = authorRepository.findAll();
for (Author a : authors) {
    System.out.println(a.getPosts().size());
}
```

This executes 1 query for authors + N queries (one per author to load `posts`) 【16†L166-L174】 .
Solutions include:

- **JOIN FETCH in JPQL:** Define a query that fetches the association:

```
@Query("SELECT a FROM Author a JOIN FETCH a.posts")
List<Author> findAllWithPosts();
```

This retrieves authors and posts in one SQL join 【16†L197-L202】 .

- **EntityGraph:** As above, annotate repository method with `@EntityGraph` to load the collection eagerly 【9†L1121-L1130】 .
- **Batch size:** Configure JPA provider batch fetching to load collections in batches (Hibernate `@BatchSize`).

Use these when you know you will traverse associations in loops or return DTOs including nested data.

Testing Strategies

Use Spring Boot's testing support for repositories:

- Annotate test classes with `@DataJpaTest` . This sets up an in-memory database (H2 by default) and configures Spring Data JPA components 【31†L84-L90】 . Example:

```
@SpringBootTest
@DataJpaTest
class UserRepositoryTest {
    @Autowired UserRepository repo;
```

```

@Test
void testFind() {
    // use repo, it will use H2 and rollback after test
}
}

```

By default `@DataJpaTest` rolls back each test's transaction, keeping DB pristine [32†L257-L260] . It also auto-creates schema (`ddl-auto = create-drop`).

- Test **derived queries** lightly (spring will validate at startup). Focus on **complex custom queries** (JPQL and native). As reflectoring points out, simple derived queries are checked for validity at startup [31†L144-L152] , but long method names and custom queries should be tested.
- For **native queries**, you often need to test against the actual database type (H2 may not support some features).
- Use real DB integration (e.g. Testcontainers) for vendor-specific SQL.
- For **Specifications/Querydsl**, tests can build specifications/predicates and verify results.

In summary, test behavior of queries rather than relying solely on Spring validation.

Migration and Compatibility Notes

- **Jakarta Namespace:** Spring Data JPA 4.x/Spring Boot 3.x migrated to Jakarta EE 9 (`jakarta.persistence.*`). Update imports if upgrading.
- **Java Version:** Spring Data JPA 4 requires Java 17+.
- **Querydsl Fork:** The Querydsl project is under a community fork (groupId `io.github.openfeign.querydsl`). Spring Data supports it on a best-effort basis [25†L100-L109] . Ensure the annotation processor matches (use `querydsl-apt`).
- **New Features:** Spring Data 4.x introduced advanced scroll queries (Window, ScrollPosition) and keyset pagination. Older versions lack `Window<T>` [20†L179-L187] .
- **Deprecations:** No major query-style deprecations, but review release notes if jumping multiple versions [13†L93-L100] .

Always consult the upgrade documentation and release notes for detailed changes.

Code Examples

Below are annotated code snippets in a Spring Boot context (Java 17+). Assume dependencies:

```

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    runtimeOnly 'com.h2database:h2' // In-memory DB for simplicity
}

```

```

        implementation 'com.querydsl:querydsl-jpa' // For Querydsl (plus
annotation processor)
    }

```

```

// Sample entity classes
@Entity
class User {
    @Id @GeneratedValue Long id;
    String firstname, lastname;
    @ManyToMany(fetch = FetchType.LAZY)
    List<Role> roles;
    // getters/setters
}

@Entity
class Role {
    @Id @GeneratedValue Long id;
    String name;
    // getters/setters
}

```

Derived Query Example:

```

public interface UserRepository extends JpaRepository<User, Long> {
    // Derived query: find by lastname AND firstname
    List<User> findByLastnameAndFirstname(String lastname, String firstname);
    // The above translates to "SELECT u FROM User u WHERE u.lastname = ?1
    AND u.firstname = ?2" [1†L131-L134] .
}

```

@Query JPQL Example with Named Parameters:

```

public interface UserRepository extends JpaRepository<User, Long> {
    @Query("SELECT u FROM User u WHERE u.firstname = :fn OR u.lastname
= :ln")
    List<User> findByFirstnameOrLastname(@Param("fn") String firstname,
                                         @Param("ln") String lastname);
    // Named parameters ensure correct binding [24†L687-L694] .
}

```

Native SQL Query Example:

```

public interface OrderRepository extends JpaRepository<Order, Long> {
    @Query(value = "SELECT o.* FROM orders o WHERE o.customer_id = :cid",
nativeQuery = true)
    List<Order> findByCustomerNative(@Param("cid") Long customerId);
}

```

```

    // NOTE: Hibernate does not validate native SQL, so test this carefully
    [31†L198-L202] .
}

```

Criteria API Example:

```

@Repository
public class ProductDao {
    @PersistenceContext
    private EntityManager em;

    public List<Product> findByCategoryAndPrice(String category, double
minPrice) {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Product> cq = cb.createQuery(Product.class);
        Root<Product> root = cq.from(Product.class);
        Predicate catPred = cb.equal(root.get("category"), category);
        Predicate pricePred = cb.greaterThan(root.get("price"), minPrice);
        cq.select(root).where(cb.and(catPred, pricePred));
        return em.createQuery(cq).getResultList();
    }
    // Returns products where category = ? and price > ?.
}

```

Querydsl Example:

```

interface UserRepository extends JpaRepository<User, Long>,
QuerydslPredicateExecutor<User> {}

// In a service or test:
QUser user = QUser.user;
Predicate pred = user.firstname.equalsIgnoreCase("Alice")
                .and(user.roles.any().name.eq("ADMIN"));
List<User> admins = (List<User>) userRepository.findAll(pred);
// Finds users named Alice with an ADMIN role [25†L131-L139] [25†L143-L146] .

```

Specifications Example:

```

public class CustomerSpecs {
    public static Specification<Customer> hasActiveTrue() {
        return (root, query, cb) -> cb.isTrue(root.get("active"));
    }
    public static Specification<Customer> hasMinPurchases(int min) {
        return (root, query, cb) -> cb.ge(root.get("purchases"), min);
    }
}

// Usage in a service or test:

```

```
List<Customer> vip = customerRepository.findAll(
    Specification.where(CustomerSpecs.hasActiveTrue())
        .and(CustomerSpecs.hasMinPurchases(100))
);
// Finds customers where active = true AND purchases >= 100 [4†L153-L162] .
```

Query by Example Example:

```
Person probe = new Person();
probe.setFirstname("Bob");
// ignore null fields, use default matching (exact for non-strings, substring
// for strings)
Example<Person> example = Example.of(probe);

List<Person> results = personRepository.findAll(example);
// Finds all persons with firstname = "Bob" (case and exact matching by
// default) [3†L175-L180] .
```

EntityGraph Example:

```
@Entity
@NamedEntityGraph(name = "User.roles",
    attributeNodes = @NamedAttributeNode("roles"))
class User { ... }

interface UserRepository extends JpaRepository<User, Long> {
    @EntityGraph(value = "User.roles", type = EntityGraphType.LOAD)
    Optional<User> findById(Long id);
    // This will fetch the roles collection with the User in one query
    [9†L1121-L1130] .
}
```

```
// Ad-hoc EntityGraph without named graph:
interface UserRepository extends JpaRepository<User, Long> {
    @EntityGraph(attributePaths = {"roles"})
    List<User> findByActiveTrue();
    // Equivalent effect: fetch 'roles' with the query [9†L1139-L1145] .
}
```

Projections Example:

```
// Interface-based projection
public interface NamesOnly {
    String getFirstname();
    String getLastname();
}
```

```

interface PersonRepo extends JpaRepository<Person, Long> {
    List<NamesOnly> findByLastname(String lastname);
}

// Using it:
List<NamesOnly> names = personRepo.findByLastname("Smith");
// Returns list of proxy objects exposing only getFirstname(), getLastName
// [5†L139-L147] .

// Nested projection interface
interface PersonSummary {
    String getFirstname();
    String getLastName();
    AddressSummary getAddress();
    interface AddressSummary { String getCity(); }
}
List<PersonSummary> summaries = personRepo.findAllProjectedBy();
// Fetches firstname, lastname, and address.city [5†L173-L182] .

```

```

// Class-based DTO projection with constructor
public class UserDTO {
    private String name;
    public UserDTO(String name) { this.name = name; }
    public String getName() { return name; }
}

interface UserRepo extends JpaRepository<User, Long> {
    @Query("SELECT new com.example.UserDTO(u.firstname) FROM User u WHERE u.active = true")
    List<UserDTO> findActiveUserNames();
}

```

Pagination/Sorting Example:

```

interface UserRepo extends JpaRepository<User, Long> {
    Page<User> findByLastname(String lastname, Pageable pageable);
    List<User> findByAgeGreaterThan(int age, Sort sort);
}

// Usage:
Page<User> page1 = userRepo.findByLastname("Doe", PageRequest.of(0, 10, Sort.by("firstname")));
List<User> sortedByAge = userRepo.findByAgeGreaterThan(18, Sort.by("age").descending());
// Page<T> requires Pageable, returns content + pagination info [20†L179-L187] .

```

Bulk Update Example (@Modifying):

```
interface UserRepo extends JpaRepository<User, Long> {
    @Modifying
    @Transactional
    @Query("UPDATE User u SET u.active = false WHERE u.lastLogin < :cutoff")
    int deactivateInactive(@Param("cutoff") LocalDate cutoff);
    // Marks old accounts inactive; @Modifying flips readOnly to false
    [29†L175-L184] .
}
```

Locking Example:

```
interface AccountRepo extends JpaRepository<Account, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT a FROM Account a WHERE a.id = :id")
    Optional<Account> findByIdWithLock(@Param("id") Long id);
    // Acquires DB lock to prevent concurrent updates [11†L93-L102] .
}
```

Feature Comparison Table

Query Style	Capabilities	Complexity	Performance	Typical Use-Cases
Derived Methods	Simple filters by properties (AND, OR, >, <, Like, etc.) [39†L147-L156]	Very low	Good for simple indexed queries	Quick finds by entity fields
@Query (JPQL)	Arbitrary JPQL (joins, aggregates) [31†L168-L172]	Medium	Good, portable SQL; can use fetch joins	Complex fixed queries, joins, projections
Native SQL	Any SQL (DB-specific features)	High	Potentially fastest (leverages DB features)	Vendor-specific queries, performance tuning
Criteria API	Dynamic queries, type-safe predicates (Metamodel) [27†L289-L297]	High (code)	Reasonable (typed, compile-checked)	Dynamic search filters, compile-time safety
Querydsl	Type-safe fluent queries (Predicate) [25†L131-L139]	Medium-High (setup)	High (fluent API, indexes used normally)	Complex dynamic queries, multi-criteria

Query Style	Capabilities	Complexity	Performance	Typical Use-Cases
Specifications	Composable Criteria predicates [4†L153-L162]	Medium (boilerplate)	Similar to Criteria API	Flexible filtering (e.g. search screens)
Query by Example	Probe-based matching (exact or patterned) [3†L175-L180]	Low	Moderate (limited to simple ops)	Simple UI searches, no code writing
EntityGraph	Eager fetch plans for relationships [9†L1121-L1130]	Low (annotation)	Greatly reduces N+1 overhead if used correctly	Preventing N+1, retrieving related entities
Projections	Partial results (DTOs/interfaces) [5†L139-L147]	Low-Medium	Improves performance by selecting few columns	API responses, reports, optimization
Pagination/Sorting	Chunked fetch (Page , Slice), dynamic Sort	Low	Controls data volume per query	UI tables, REST list endpoints

Query Style Decision Flow

flowchart TD

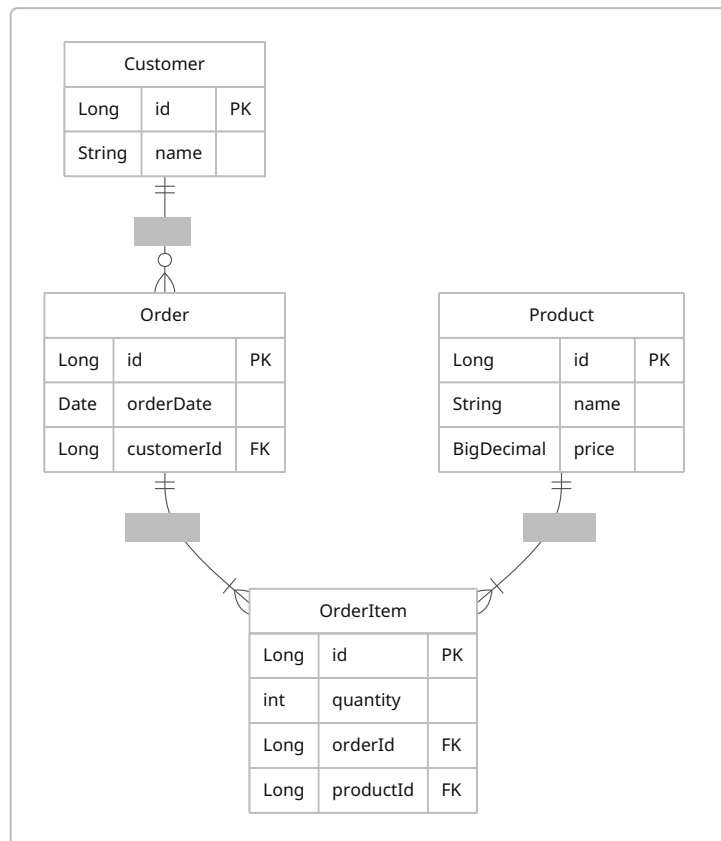
```

A[Determine Query Requirement] --> B{Static vs Dynamic?}
B -- Static and Simple --> C[Derived Method]
B -- Static and Complex --> D[@Query (JPQL)]
B -- Dynamic / Many Filters --> E{Type-safe needed?}
E -- Yes --> F[Querydsl or Criteria/Specification]
E -- No / Quick --> G[Specifications or Query by Example]
C --> H{Partial fields?}
D --> H
H -- Yes --> I[Projections (DTO/Interface)]
H -- No --> J[Return Entity]
F --> K[Use QuerydslPredicateExecutor or Criteria API]
G --> K
K --> L{Performance critical?}
L -- Yes (DB features needed) --> M[Native Query / Stored Proc]
L -- No --> J

```

This flow guides choosing a query style based on requirements: use **Derived methods** for simple fixed queries, [@Query](#) for complex static queries, **Querydsl/Criteria/Specifications** for dynamic logic, and **Projections** if only partial data is needed. For raw performance or DB-specific needs, use **native SQL**.

Entity Relationship Example Diagram



This ER diagram shows Customers placing Orders, Orders having OrderItems linking to Products.

Sources: Official Spring Data JPA documentation and guides [【1†L131-L134】](#) [【24†L687-L694】](#) [【4†L153-L162】](#) [【5†L139-L147】](#) [【9†L1121-L1130】](#) [【11†L93-L102】](#) [【12†L93-L100】](#) [【20†L179-L187】](#) , plus reputable tutorials [【31†L84-L90】](#) [【36†L259-L268】](#) . Code examples are illustrative and ready to use in a Spring Boot project (add appropriate dependencies and `@Entity` classes). Adjust package names and imports (e.g. `jakarta.persistence.*`) according to your setup.
